

Document Intelligence RAG: Complete Project Guide

1. What this project does

This is a Streamlit application for asking questions about uploaded documents. It supports PDF, DOCX, TXT, Markdown, CSV, and XLSX files.

The application uses:

- Google Gemini for document analysis, query analysis, answer generation, and evaluation.
- Jina Embeddings for document and question vectors.
- ChromaDB for local vector search.
- Pydantic models for predictable data contracts.
- Streamlit for the user interface.

The central idea is Retrieval-Augmented Generation (RAG): the application retrieves relevant document chunks first, then gives only those chunks to Gemini so the answer stays grounded in the uploaded files.

2. Project files

Root files

- ``app.py``: Streamlit entry point and user interface. It initializes the pipeline, accepts uploads, runs document processing, handles chat, displays sources, and runs evaluation.
- ``requirements.txt``: Python packages required to run the application.
- ``env.example``: Template for API keys and runtime settings. Copy it to ``env`` and fill in the keys.
- ``README.md``: Short project overview, setup instructions, architecture, and feature summary.
- ``gitignore``: Files that should not be committed, such as secrets, virtual environments, caches, and generated data.

Source files

- ``src/__init__.py``: Marks ``src`` as a Python package.
- ``src/models.py``: Pydantic models shared by every stage. It defines raw pages, raw documents, analyses, metadata, chunks, query analysis, retrieval results, citations, RAG responses, and evaluation metrics.
- ``src/prompts.py``: All Gemini prompt templates. Keeping prompts here makes them easy to review and change.
- ``src/llm.py``: The single Gemini model factory. It reads ``GEMINI_API_KEY`` and ``GEMINI_MODEL``, validates the key, and returns a deterministic chat model.
- ``src/jina_embeddings.py``: Small HTTP client for the Jina embeddings API. It embeds document text and user questions.

- ``src/embeddings.py``: Simple embedding factory that always returns ``JinaEmbeddings``.
- ``src/document_loader.py``: Converts each supported file format into a ``RawDocument`` made of pages or sheets.
- ``src/document_analyzer.py``: Sends the first part of the extracted text to Gemini to infer type, topic, purpose, sections, entities, dates, organizations, people, and a summary.
- ``src/parser.py``: Detects headings and turns pages into ``ParsedSection`` objects. It also preserves tables as separate sections.
- ``src/chunker.py``: Creates searchable chunks. Structure-aware mode keeps sections and tables meaningful; naive mode provides a simple baseline.
- ``src/vector_store.py``: Adds chunks to ChromaDB, performs similarity search, exposes metadata, counts chunks, and resets the collection.
- ``src/query_analyzer.py``: Uses Gemini to understand the question and identify possible document-type filters and keywords.
- ``src/retriever.py``: Combines query analysis, metadata filtering, similarity search, fallback search, ranking, and top-k selection.
- ``src/rag_chain.py``: Formats retrieved chunks and recent conversation history, invokes Gemini, extracts source citations, and returns a ``RAGResponse``.
- ``src/evaluation.py``: Generates test questions and uses Gemini as a judge for retrieval relevance, faithfulness, and completeness. Citation correctness is checked in Python.

Data and documentation

- ``data/uploads/``: Intended location for uploaded-file related data. The current Streamlit upload flow processes files in memory.
- ``data/uploads/.gitkeep``: Keeps the empty upload directory in source control.
- ``docs/PROJECT_GUIDE.md``: This complete written guide.
- ``generate_project_pdf.py``: Converts this guide into a readable PDF.
- ``docs/PROJECT_GUIDE.pdf``: Generated PDF version of this guide.

3. Configuration

Create `.env` from `.env.example`:

```
GEMINI_API_KEY=your-gemini-api-key-here
GEMINI_MODEL=gemini-3.6-flash
JINA_API_KEY=your-jina-api-key-here
JINA_EMBEDDING_MODEL=jina-embeddings-v5-text-small
CHUNK_SIZE=1000
CHUNK_OVERLAP=200
TOP_K=5
```

The keys are used as follows:

- ``GEMINI_API_KEY``: authenticates every Gemini request.
- ``GEMINI_MODEL``: selects the Gemini model used by all LLM stages.
- ``JINA_API_KEY``: authenticates document and query embedding requests.

- ``JINA_EMBEDDING_MODEL``: selects the Jina embedding model.
- ``CHUNK_SIZE``: maximum target size for regular chunks.
- ``CHUNK_OVERLAP``: overlap used when long sections are split.
- ``TOP_K``: number of chunks returned to the answer stage.

4. Installation and startup

From the project directory:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
# Edit .env and add both API keys
streamlit run app.py
```

Open the local Streamlit URL shown in the terminal. The application needs both API keys when the pipeline is initialized.

5. Document ingestion pipeline

The upload pipeline in `app.py` runs these steps:

```
Uploaded file
-> DocumentLoader.load
-> DocumentAnalyzer.analyze and generate_summary
-> DocumentParser.parse
-> DocumentChunker.chunk_structure_aware or chunk_naive
-> JinaEmbeddings
-> ChromaDB
-> Streamlit session state
```

Step 1: Upload

The sidebar accepts PDF, DOCX, TXT, MD, CSV, and XLSX files. `process_document` reads the uploaded bytes into memory and passes the file name plus a byte stream to the loader.

Step 2: Load

`DocumentLoader` chooses a loader from the file extension:

- PDF: ``pypdf`` extracts one ``RawPage`` per PDF page.
- DOCX: ``python-docx`` extracts paragraphs, heading markers, list markers, and table rows. DOCX is represented as one logical page because it has no simple native page boundary in this implementation.
- TXT and MD: decoded as UTF-8 and represented as one page.
- CSV: read with `pandas`, rendered as a pipe-delimited table, and represented as one page.
- XLSX: every non-empty worksheet becomes one logical page with sheet name, columns, row count, and table text.

The loader produces `RawDocument`, which contains the file name, file type, pages, and page count.

Step 3: Analyze with Gemini

DocumentAnalyzer concatenates extracted pages and sends at most 8,000 characters to Gemini. The prompt asks for JSON containing document type, topic, purpose, language, composition, sections, entities, dates, organizations, people, summary, and confidence.

If Gemini fails or returns invalid JSON, the analyzer returns a small fallback analysis instead of stopping all ingestion.

Step 4: Parse structure

DocumentParser detects Markdown headings, uppercase heading-like lines, and numbered headings. It creates ParsedSection objects with title, level, content, page number, and table status. Tables are added as separate sections so they can remain intact during chunking.

Step 5: Chunk

The default structure-aware strategy creates one chunk per short section, keeps tables atomic, and uses a recursive text splitter for long sections. The naive strategy concatenates all content and splits it without section awareness. Every chunk receives document ID, file name, document type, page number, section, and chunking strategy.

Step 6: Embed with Jina

VectorStore.add_chunks passes chunk text to ChromaDB. ChromaDB calls JinaEmbeddings.embed_documents, which sends the texts to the Jina API and receives numeric vectors.

Step 7: Store

ChromaDB stores the vectors, text, and flattened metadata. The same embedding model is later used for the user's query, which makes document vectors and query vectors comparable.

6. Question and answer pipeline

When a user submits a question, RAGChain.query runs:

```
Question
-> QueryAnalyzer with Gemini
-> metadata filter construction
-> ChromaDB similarity search using Jina query embedding
-> fallback search if filtering returns nothing
-> top-k RetrievalResult objects
-> context formatting with source labels
-> Gemini answer generation
-> source citation extraction
-> conversation history update
```

Query analysis

QueryAnalyzer receives the question and a list of uploaded document summaries. Gemini returns intent, target document types, target sections, multi-document status, comparison status, and keywords. Retriever uses matching document types to build a ChromaDB filter.

Retrieval

The retriever searches for TOP_K + 3 chunks, then keeps the best TOP_K. If a metadata filter produces no result, it retries without the filter. Each result keeps its text, metadata, relevance score, and rank.

Grounded generation

RAGChain builds a context block with file name, page, section, and document type. The RAG prompt tells Gemini to use only this context, avoid invented facts, state when information is missing, and cite sources.

The last ten exchanges are included as short conversation history. The history supports follow-up questions, but the retrieved document context remains the factual source.

7. Evaluation pipeline

The Evaluation tab uses RAGEvaluator:

1. Gemini generates questions and expected answers from each uploaded document.
2. Each generated question runs through the normal retrieval and answer path.
3. Gemini scores retrieval relevance, answer faithfulness, and answer completeness.
4. Context relevance currently mirrors retrieval relevance.
5. Python checks whether every cited file exists among retrieved chunks.
6. The UI shows averages and per-question details.

8. User interface tabs

- Chat: ask questions and view answers with citations.
- Documents: see document count, chunk count, types, and processing status.
- Document Analysis: inspect Gemini's inferred type, topic, summary, sections, entities, dates, organizations, and people.
- Sources: inspect query understanding, retrieved chunks, metadata, and relevance scores.
- Evaluation: generate and run quality checks.

9. Important design decisions

- Provider setup is centralized in ``llm.py`` and ``embeddings.py``, so runtime modules do not each implement credentials or model selection.
- Pydantic models make the data shape explicit between pipeline stages.

- Structure-aware chunks improve retrieval around headings and tables.
- Metadata is flattened before ChromaDB storage because vector-store metadata must be simple values.
- The retriever has a graceful unfiltered fallback for imperfect query classification.
- The application currently uses in-memory Streamlit state and a non-persistent Chroma collection, so a restart clears the working session.

10. Known limitations

- Scanned PDFs need OCR, which is not included.
- Very large documents are truncated during AI analysis.
- The current upload flow is single-user and session-based.
- DOCX page numbers are approximated as one logical page.
- Retrieval uses dense similarity only; no keyword hybrid search or reranker is included.
- API availability, latency, and cost depend on Gemini and Jina service access.

11. One-file summary

app.py	UI and orchestration
models.py	typed data contracts
prompts.py	Gemini instructions
llm.py	Gemini factory
jina_embeddings.py	Jina HTTP embeddings client
embeddings.py	Jina embedding entry point
document_loader.py	file extraction
document_analyzer.py	AI classification and summary
parser.py	structure detection
chunker.py	searchable chunk creation
vector_store.py	ChromaDB storage and search
query_analyzer.py	question understanding
retriever.py	filtered vector retrieval
rag_chain.py	grounded answer generation
evaluation.py	RAG quality evaluation